

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра МОЭВМ**

**ЛАБОРАТОРНАЯ РАБОТА №2**  
**по дисциплине «Вычислительная математика»**  
**Тема: решение систем нелинейных уравнений**

Студент гр. 4341

Четвертная В.Л.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

### **Цель работы.**

Целью лабораторной работы являются изучение прямых (метод Гаусса с выбором главного элемента по столбцу) и итерационных (метод Якоби) методов решения систем линейных алгебраических уравнений (СЛАУ), написание программ, реализующих данные методы, и исследование зависимости числа итераций от заданной точности (для метода Якоби), а также исследование обусловленности вычислительных алгоритмов (чувствительности к ошибкам округления и исходным данным).

### **Задание.**

Требуется разработать программы для решения СЛАУ вида  $Ax=b$  с матрицами различной размерности:

1. Реализовать метод Гаусса с выбором главного элемента по столбцу. Проверить корректность решения сравнением с результатами библиотечной функции LAPACKE\_dgesv (эталон). Оценить погрешность в различных нормах.
2. Составить подпрограмму-генератор исходных данных (генерация матрицы коэффициентов и вектора правых частей). Для итерационных методов обеспечить диагональное преобладание в матрице.
3. Реализовать метод Якоби. Провести вычисления, исследуя зависимость числа итераций от заданной точности  $\epsilon$ .

## **Выполнение работы.**

Исходные данные моделируются с использованием генератора псевдослучайных чисел (равномерное распределение). Для итерационного метода Якоби матрица модифицируется для обеспечения строгого диагонального преобладания ( $|a_{ii}| > \sum |a_{ij}|, i \neq j$ ).

Разработка программы.

Программы написаны на языке C++ с использованием библиотеки линейной алгебры LAPACK.

Генерация данных: Отдельная программа для сохранения матрицы в файл, либо встроенные процедуры заполнения случайными числами (диапазон  $[0, 10]$  или  $[0, 1000]$ ).

Функция вычисления погрешности: реализованы `infNorm`, `singleNorm`, `doubleNorm` для оценки относительной погрешности в различных нормах.

Метод Гаусса.

Функция `int gauss(int n)`. Содержит прямой ход с частичным выбором ведущего элемента по столбцу. Перестановка строк массива реализована сменой указателей через `std::swap`.

Обратный ход реализован стандартным способом. Возврат кода ошибки (1), если ведущий элемент равен нулю.

Метод Якоби.

Функция `int jacoby(int n, double e)`. Содержит итерационный цикл `while (diff >= e)`. Вычисление очередного приближения происходит по формуле  $x_i^{(k+1)} = (b_i - \sum a_{ij} * x_j^{(k)}) / a_{ii}$ .

Критерий остановки — выполнение условия максимального покомпонентного отклонения меньше заданного  $e$ .

## Тестирование метода Гаусса.

Матрицы Гильберта

5\*5

Gauss: 0

Lapacke: 0

Infini norm: 0.000000000000003446141

Single norm: 0.000000000000003486712

Double norm: 0.00000024127204622000

10\*10

Gauss: 0

Lapacke: 0

Infini norm: 0.00000000191304556119

Single norm: 0.00000000244430894041

Double norm: 0.00006704079729016231

100\*100

Gauss: 0

Lapacke: 0

Infini norm: 0.00000003566211607282

Single norm: 0.00000003560375037664

Double norm: 0.00020894359686380779

500\*500

Gauss: 0

Lapacke: 0

Infini norm: 0.00000006910430463417

Single norm: 0.00000006838045469778

Double norm: 0.00035234919808739215

1000\*1000

Gauss: 0

Lapacke: 0

Infini norm: 0.00000004342493761650

Single norm: 0.00000004146042843953

Double norm: 0.00023266212908272741

Случайная матрица 1000\*1000

Gauss: 0

Lapacke: 0

Infini norm: 0.00000000000058880780

Single norm: 0.00000000000054591345

Double norm: 0.00000101623205507752

## **Тестирование метода Якоби.**

500 3

Generate done

It: 1

Jacoby: 0

Lapacke: 0

Infini norm: 0.34438594793010096629

Single norm: 0.28569119736343134841

Double norm: 0.65718729541507414638

1000 3

Generate done

It: 1

Jacoby: 0

Lapacke: 0

Infini norm: 0.33475253267399068813

Single norm: 0.27516768252711437670

Double norm: 0.64170242228956864139

1000 6

Generate done

It: 1

Jacoby: 0

Lapacke: 0

Infini norm: 0.33516230492226617432

Single norm: 0.28061249956744188649

Double norm: 0.66191224976471796371

5000 3

Generate done

It: 1

Jacoby: 0

Lapacke: 0

Infini norm: 0.33437009340126372381

Single norm: 0.27953495039781062204

Double norm: 0.65226010826494362771

5000 6

Generate done

It: 1

Jacoby: 0

Lapacke: 0

Infini norm: 0.33262604402425488548

Single norm: 0.28429920533568509011

Double norm: 0.65210358347481645058

### **Выводы.**

В ходе выполнения лабораторной работы были изучены и реализованы методы решения СЛАУ: метод Гаусса с частичным поворотом и метод Якоби. Метод Якоби работает за гораздо меньшее число итераций, однако даёт большую погрешность и подходит только для матриц с диагональным преобладанием.



## ПРИЛОЖЕНИЕ А.

### ИСХОДНЫЙ КОД ПРОГРАММЫ

```
#include <lapacke.h>
#include <iostream>
#include <iomanip>
#include <fstream>
#include <random>
#include <vector>
#include <cmath>

double a[10000][10000] = {{0}};
double b[10000]={0}, b1[10000]={0};
double row[100000000]={0};
double x[10000]={0};

int gauss(int n){
    if (n < 2)
        return 1;

    for (int k=0; k < n-1; k++){
        int maxInd=k;
        double maxVal=fabs(a[k][k]);

        for (int i=k+1; i < n; i++){
            if (fabs(a[i][k]) > maxVal){
                maxVal = fabs(a[i][k]);
                maxInd = i;
            }
        }

        if (maxVal == 0.0)
            return 1;

        if (maxInd != k) {
            std::swap(a[maxInd], a[k]);
            std::swap(b[maxInd], b[k]);
        }

        for (int i=k+1; i < n; i++){
            double mi=a[i][k]/a[k][k];
            a[i][k] = 0.0;

            for (int j=k+1; j < n; j++)
                a[i][j] -= a[k][j]*mi;

            b[i] -= b[k]*mi;
        }

        std::cout << "Line " << k << " done\n";
    }

    if (a[n-1][n-1] == 0.0)
        return 1;

    for (int i=n-1; i >= 0; i--){
        double sum=0.0;
```

```

        for (int j=i+1; j < n; j++)
            sum += a[i][j]*x[j];

        x[i] = (b[i]-sum)/a[i][i];
    }

    return 0;
}

double infNorm(int n, double x_expect[], double x_reciev[]){
    double result=0;
    double expect=0;

    for (int i=0; i < n; i++){
        result = std::max(result, fabs(x_expect[i]-x_reciev[i]));
        expect = std::max(expect, fabs(x_expect[i]));
    }

    return result/expect;
}

double singleNorm(int n, double x_expect[], double x_reciev[]){
    double result=0.0;
    double expect=0.0;

    for (int i=0; i < n; i++){
        result += fabs(x_expect[i]-x_reciev[i]);
        expect += fabs(x_expect[i]);
    }

    return result/expect;
}

double doubleNorm(int n, double x_expect[], double x_reciev[]){
    double result=0.0;
    double expect=0.0;

    for (int i=0; i < n; i++){
        result += fabs(x_expect[i]*x_expect[i]-
x_reciev[i]*x_reciev[i]);
        expect += fabs(x_expect[i]*x_expect[i]);
    }

    result = std::sqrt(result);
    expect = std::sqrt(expect);

    return result/expect;
}

int main(){
    int n;
    std::cin >> n;

    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution <double> dis(0.0, 10.0);

    for (int i=0; i < n; i++){

```

```

        for (int j=0; j < n; j++){
            a[i][j] = dis(gen);
            row[n*i+j] = a[i][j];
        }
        b[i] = dis(gen);
        b1[i] = b[i];
    }

    std::cout << "Generate done\n";

    int res=gauss(n);
    std::cout << "Gauss: " << res << "\n";
    // for (int i=0; i < n; i++)
    //     std::cout << x[i] << " ";
    // std::cout << "\n";

    std::vector <int> ipiv(n);
    int info=LAPACKE_dgesv(LAPACK_ROW_MAJOR, n, 1, row, n,
ipiv.data(), b1, 1);
    std::cout << "Lapacke: " << info << "\n";

    double inf=infNorm(n, b1, x), single=singleNorm(n, b1, x),
dob=doubleNorm(n, b1, x);
    std::cout << std::fixed << std::setprecision(20) << "Infini
norm: " << inf << "\nSingle norm: " << single << "\nDouble norm: " << dob
<< "\n";
    }

#include <iostream>
#include <fstream>
#include <iomanip>
#include <vector>
#include <random>

int main(){
    std::ofstream out("input.txt");

    int n;
    std::cin >> n;

    out << n << "\n";

    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution <double> dis(0.0, 1000.0);

    for (int i=0; i < n; i++){
        for (int j=0; j < n+1; j++){
            // double a=1.0/(1+j+i);
            // out << std::fixed << std::setprecision(10) << a << "
";
            out << dis(gen) << " ";
        }
        out << "\n";
    }

    out.close();
}

```

```

#include <lapacke.h>
#include <iostream>
#include <climits>
#include <iomanip>
#include <fstream>
#include <random>
#include <cmath>

double a[10000][10000] = {{0}};
double b[10000]={0}, add[10000]={0};
double row[100000000]={0};
double x[10000]={0};

double infNorm(int n, double x_expect[], double x_reciev[]){
    double result=0;
    double expect=0;

    for (int i=0; i < n; i++){
        result = std::max(result, fabs(x_expect[i]-x_reciev[i]));
        expect = std::max(expect, fabs(x_expect[i]));
    }

    return result/expect;
}

double singleNorm(int n, double x_expect[], double x_reciev[]){
    double result=0.0;
    double expect=0.0;

    for (int i=0; i < n; i++){
        result += fabs(x_expect[i]-x_reciev[i]);
        expect += fabs(x_expect[i]);
    }

    return result/expect;
}

double doubleNorm(int n, double x_expect[], double x_reciev[]){
    double result=0.0;
    double expect=0.0;

    for (int i=0; i < n; i++){
        result += fabs(x_expect[i]*x_expect[i]-
x_reciev[i]*x_reciev[i]);
        expect += fabs(x_expect[i]*x_expect[i]);
    }

    result = std::sqrt(result);
    expect = std::sqrt(expect);

    return result/expect;
}

int jacoby(int n, double e){
    if (n < 1)
        return 1;

```

```

int it=0;
double diff=e+1;

while (diff >= e){
    diff = 0;

    for (int i=0; i < n; i++){
        double sum=0, next_el=0;
        for (int j=0; j < n; j++){
            if (j != i)
                sum += a[i][j]*x[j];
        }
        next_el = (b[i]-sum)/a[i][i];

        diff = std::max(diff, fabs(x[i]-next_el));
        x[i] = next_el;
    }
    it++;
}

std::cout << "It: " << it << "\n";
return 0;
}

int main(){
    int n;
    double e;
    std::cin >> n >> e;

    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution <double> dis(0.0, 10.0);
    std::uniform_real_distribution <double> delt(1.0, 2.0);

    for (int i=0; i < n; i++){
        double sum=0;
        for (int j=0; j < n; j++){
            a[i][j] = dis(gen);
            row[n*i+j] = a[i][j];

            sum += a[i][j]*(i != j);
        }

        a[i][i] = sum+delt(gen);
        row[n*i+i] = a[i][i];

        b[i] = dis(gen);
        add[i] = b[i];
    }

    std::cout << "Generate done\n";

    int res=jacoby(n, e);
    std::cout << "Jacoby: " << res << "\n";
    // for (int i=0; i < n; i++)
    //     std::cout << x[i] << " ";
    // std::cout << "\n";

```

```

        std::vector<int> ipiv(n);
        int info=LAPACKE_dgesv(LAPACK_ROW_MAJOR, n, 1, row, n,
ipiv.data(), add, 1);
        std::cout << "Lapacke: " << info << "\n";

        double inf=infNorm(n, add, x), single=singleNorm(n, add, x),
dob=doubleNorm(n, add, x);
        std::cout << std::fixed << std::setprecision(20) << "Infini
norm: " << inf << "\nSingle norm: " << single << "\nDouble norm: " << dob
<< "\n";
    }

```